

DL Midsem Notes — Section 3

McCulloch–Pitts Neuron and Thresholding Logic

1 Theory

1.1 The MP Neuron Model

Inputs are binary: $x_i \in \{0, 1\}$. Inputs are either **excitatory** (weight $+1$) or **inhibitory**. The neuron fires (outputs 1) if the sum of excitatory inputs meets or exceeds a threshold θ , **unless any inhibitory input is active**, in which case it outputs 0 regardless of everything else:

$$y = \begin{cases} 0 & \text{if any inhibitory input} = 1 \\ 1 & \text{if } \sum x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

All excitatory weights are fixed at 1 — there is no learning, only manual choice of θ (and which inputs are inhibitory).

1.2 Limitations of the MP Neuron

- **No learning.** Weights are fixed at ± 1 or "inhibitory"; θ must be picked by hand for every new function.
- **Only Boolean inputs.** Real-valued inputs aren't handled.
- **Only linearly separable Boolean functions.** A single MP neuron realizes a hyperplane threshold; anything not linearly separable (XOR) is out of reach.
- **Excitatory weights are all equal.** No way to weigh one input more heavily than another (only presence/absence of inhibition is asymmetric).

How the perceptron overcomes this: it replaces fixed ± 1 /inhibitory weights with *learnable, real-valued* weights w_i and a learnable bias, plus a training rule ($w \leftarrow w + \eta(y - \hat{y})x$) that adjusts them from data. This allows real-valued inputs and differently-weighted inputs — but it is still fundamentally a linear threshold device, so it still cannot solve XOR (that requires stacking neurons — section 4).

1.3 Boolean Logic Gates as MP Neurons

AND (n -bit): all inputs excitatory, weight 1 each, threshold $\theta = n$ (all n inputs must be 1). **OR** (n -bit): all inputs excitatory, weight 1 each, threshold $\theta = 1$ (at least one input 1 suffices). **NOT**: single inhibitory input; any active input forces output 0; with no input active, and threshold $\theta = 0$, output is 1.

2 Practice Questions

Q1 (PYQ-style, CS6510-1 Q1A). Explain the McCulloch–Pitts neuron along with its limitations. How does the perceptron model overcome these limitations?

Q2 (PYQ-style, CS6510-5 Q1b). Design an MP neuron to model a 3-bit logical AND problem.

Q3. Design an MP neuron for a 2-bit logical OR gate. State the weights and threshold.

Q4 (concept check). Can a single MP neuron implement a 2-input NAND gate? Justify with the threshold logic, or explain why not.

3 Answers

A1. See Theory 1.1–1.2 above for the full explanation: MP neuron = fixed-weight thresholding device with binary inputs and inhibitory override; limitations are no learning, Boolean-only, only linearly separable functions, no differential weighting. Perceptron overcomes the "no learning" and "no differential weighting" limitations via learnable real-valued weights and a training rule, but remains linear-threshold-only (can't solve XOR).

A2. Three excitatory inputs x_1, x_2, x_3 , each weight 1, threshold $\theta = 3$:

$$y = 1 \text{ iff } x_1 + x_2 + x_3 \geq 3$$

This is 1 only when all three inputs are 1 — exactly 3-bit AND.

A3. Two excitatory inputs x_1, x_2 , weight 1 each, threshold $\theta = 1$:

$$y = 1 \text{ iff } x_1 + x_2 \geq 1$$

True whenever at least one input is 1 — 2-bit OR.

A4. No. NAND is $\text{NOT}(\text{AND}) = 1$ except when both inputs are 1. Truth table: $(0,0) \rightarrow 1, (0,1) \rightarrow 1, (1,0) \rightarrow 1, (1,1) \rightarrow 0$. This requires an output that *decreases* as the sum of inputs increases past a point — but MP-neuron logic only supports "sum $\geq \theta$ turns output on" (monotonically non-decreasing in the excitatory sum), or a hard inhibitory override triggered by a *specific single input being active*, not by *both* being active simultaneously. There's no single threshold θ on $x_1 + x_2$ that produces this pattern, and no single inhibitory input can detect "both active" — inhibition here is only ever triggered by one wire being high. So NAND needs more structure than a single MP neuron (it's realizable by composing MP neurons, e.g. AND followed by NOT).